

Lecture 5 - Inductive data types and functional programming

Drawing on functional programming, Dafny enables users to define new data types; among these, inductive data types are permitted, which are recursively defined in terms of themselves, as long as the recursion is well-founded. We will see how to program and reason about the values of such types.

Inductive data types

High-level programming languages support user-defined data types. An example is finite, often called enumerated types, such as the following color type.

```
datatype Color = Blue | Yellow | Green | Red
```

Generally, a data type is defined by introducing a finite set of symbols called constructors, each depending on a finite number of arguments. In the case of Color, the constructors are all nullary, meaning they do not depend on any arguments, and can be seen as the generators of the set of values of type Color.

More interesting cases are inductive data types, which are types with a recursive definition where each reference to the type being defined always appears as an argument of a constructor. A well-known example is Peano's unary definition of natural numbers.

```
datatype Unary = Zero | Suc(pred: Unary)
```

Despite referencing itself, this is not a circular definition; in fact, the values of type Unary are generated starting from Zero and closed under the application of Suc.

```
Zero, Suc(Zero), Suc(Suc(Zero)), Suc(Suc(Suc(Zero))), ...
```

The "pred" in the definition of Suc is simply the name of the field in the internal representation of the Suc constructor, so that if $u == \text{Suc}(v)$, then $u.\text{pred} == v$. The predecessor function, often called a *destructor*, can be defined as follows, although in a more elaborate way. First, we need to consider that the predecessor is undefined for the value Zero, which can be handled using the predicate.

```

predicate IsZero(n: Unary)
{
    match n
    case Zero => true
    case Suc(_) => false
}

```

Then we use the negation of the predicate `IsZero` as the precondition of the function `Pred`:

```

function Pred(n: Unary): Unary
    requires !IsZero(n)
    ensures  Pred(n) == n.pred
{
    match n
    case Suc(m) => m
}

```

The essential fact about the predecessor is enunciated and proved in the following lemma:

```

lemma PredSuc(n: Unary)
    ensures Pred(Suc(n)) == n
{}

```

It is clear that the values of type `Unary` are bijective with the natural numbers, specifically, the values of Dafny's built-in type `nat`. We define this bijection by specifying the functions `UnaryToNat(u)` and `NatToUnary(x)`, which are inverses of each other; we also demonstrate the matching mechanism in Dafny, which is typical of functional programming languages.

```

function UnaryToNat(u: Unary): nat
{
    match u
    case Zero => 0
    case Suc(u) => 1 + UnaryToNat(u)
}

```

The meaning of the above match-based definition is the same as the equational definition you would see in a Haskell program, for example, that is (omitting types):

```

UnaryToNat(Zero)    = 0
UnaryToNat(Suc(u)) = 1 + UnaryToNat(u)

```

The inverse function w.r.t. `UnaryToNat` is

```
function NatToUnary(x: nat): Unary
{
    if x == 0 then Zero
    else Suc(NatToUnary(x - 1))
}
```

Now, we verify with the following lemma that UnaryToNat and NatToUnary are inverse of each other (the proof is entirely automatic):

```
lemma NatUnaryCorrespondence(n: nat, x: Unary)
    ensures UnaryToNat(NatToUnary(n)) == n
    ensures NatToUnary(UnaryToNat(x)) == x
{}
```

The primary tool is structural induction when defining functions (or methods) with inductive data types and reasoning about them. Given values of type Unary, we can order them by defining $t \leq t'$ whenever t is a subexpression of t' (possibly equal to it). This relation is a partial order (and is also total in the case of Unary), called structural order, which is well-founded, meaning no infinite sequence $t_0 > t_1 > t_2 > \dots$ exists.

Structural induction has been used both in the definitions of UnaryToNat and NatToUnary and in the proof of the lemma NatUnaryCorrespondence. Indeed, Dafny uses the structural order relation to verify the decrease clause (either explicit or implicit) of all functions, methods, and lemmas involving inductive data types.

As another example, consider the following definition of addition in Peano arithmetic:

```
function Add(x: Unary, y: Unary): Unary
{
    match y
    case Zero => x
    case Suc(z) => Suc(Add(x, z))
}
```

Then we demonstrate that the function Add corresponds to addition over natural numbers, by a proof using structural induction in a manner consistent with ordinary mathematical induction.

```

lemma {:induction false} AddCorrect(x: Unary, y: Unary)
  ensures UnaryToNat(Add(x, y)) == UnaryToNat(x) + UnaryToNat(y)
{
  match y
  case Zero =>
    calc {
      UnaryToNat(Add(x, y));
    == {assert y == Zero ==> Add(x, y) == x; }
      UnaryToNat(x);
    == UnaryToNat(x) + 0;
    == {assert y == Zero ==> UnaryToNat(y) == 0; }
      UnaryToNat(x) + UnaryToNat(y);
    }
  case Suc(z) =>
    calc {
      UnaryToNat(Add(x, Suc(z)));
    == UnaryToNat(Suc(Add(x, z)));
    == 1 + UnaryToNat(Add(x, z));
    == {AddCorrect(x, z);} // induction hypothesis
      1 + UnaryToNat(x) + UnaryToNat(z);
    }
}

```

Lists

Lists are data structures widely used in programming; here is an inductive definition of lists of values of a generic type T , which is expressed by the binding in List:

```
datatype List<T> = Nil | Cons(head: T, tail: List<T>)
```

According to the definition, values of type List are finite sequences of values of type T , called list elements. Functions that do not depend on the specific values in the lists can be defined with type T as a parameter, meaning they are polymorphic functions.

Consider the function Length(xs) returning the number of elements of xs:

```

function Length<T>(xs: List<T>): nat
{
  match xs
  case Nil => 0
  case Cons(_, tail) => 1 + Length(tail)
}

```

The next function Append(xs, ys) returns the new list created by concatenating the elements of xs before those of ys. Notice that values of type List are unmodifiable, so executing

Append—more accurately seen as evaluating its definition with the actual value of the parameter `xs`—has no side effects on `xs` and `ys`.

```
function Append<T>(xs: List<T>, ys: List<T>): List<T>
  ensures Length(Append(xs, ys)) == Length(xs) + Length(ys)
{
  match xs
  case Nil => ys
  case Cons(x, tail) => Cons(x, Append(tail, ys))
}
```

The post-condition `Length(Append(xs, ys)) == Length(xs) + Length(ys)` of `Append(xs, ys)` is automatically verified; to see an explicit proof, we establish the following lemma by structural induction on `xs`.

```
lemma {:induction false} AppendLength<T>(xs: List<T>, ys: List<T>)
  ensures Length(Append(xs, ys)) == Length(xs) + Length(ys)
{
  match xs
  case Nil =>
    calc {
      Length(Append(xs, ys));
    == {assert xs == Nil ==> Append(xs, ys) == ys;}
      Length(ys);
    == 0 + Length(ys);
    == {assert xs == Nil ==> Length(xs) == 0; }
      Length(xs) + Length(ys);
    }
  case Cons(x, tail) =>
    calc {
      Length(Append(Cons(x, tail), ys));
    == Length(Cons(x, Append(tail, ys)));
    == 1 + Length(Append(tail, ys));
    == {AppendLength(tail, ys);} // induction hypothesis
      1 + Length(tail) + Length(ys);
    == Length(Cons(x, tail)) + Length(ys);
    }
}
```

Sorting

To further explore verifying functional programs with Dafny, we examine the sorting problem for lists of integers, proving the correctness of a version of the quadratic algorithm, insertion sort, for lists.

```
datatype List<T> = Nil | Cons(head: T, tail: List<T>)
```

The purely functional definition of insertion sort is divided into a main function `InsertSort(xs)` and an auxiliary function `Insert(y, xs)`. `InsertSort` simply iterates `Insert` over the list to be sorted, while `Insert(y, xs)` is defined:

```
function Insert(y: int, xs: List<int>): List<int>
{
  match xs
  case Nil => Cons(y, Nil)
  case Cons(x, tail) =>
    if y < x then Cons(y, xs)
    else Cons(x, Insert(y, xs.tail))
}

function InsertSort(xs: List<int>): List<int>
{
  match xs
  case Nil => Nil
  case Cons(x, tail) => Insert(x, InsertSort(tail))
}
```

Instead of defining pre- and post-conditions for `Insert` and `InsertSort`, we use the so-called *extrinsic verification* technique. This involves introducing the predicate `Ordered(xs)` (which is marked as ghost, meaning it is not part of the actual code) and proving that the result of `InsertSort(xs)` is an ordered list for any `xs`.

```
ghost predicate Ordered(xs: List<int>)
{
  match xs
  case Nil => true
  case Cons(x, Nil) => true
  case Cons(x, Cons(y, _)) => x <= y && Ordered(xs.tail)
}
```

Before proving the main lemma, we establish a secondary one that states if `xs` is ordered, then `Insert(y, xs)` is also ordered.

```

lemma {:induction false} InsertOrdered(y: int, xs: List<int>)
  requires Ordered(xs)
  ensures  Ordered(Insert(y, xs))
{
  match xs
  case Nil => assert xs == Nil ==> Ordered(xs);
  case Cons(x, tail) =>
    if y < x {
      calc {
        Ordered(Insert(y, Cons(x, tail)));
      }
      <==> Ordered(Cons(y, Cons(x, tail)));
      <==> y <= x && Ordered(tail);
    }
  } else {
    assert x <= y;
    calc {
      true;
    }
    ==> { assert Ordered(Cons(x, tail)) ==> Ordered(tail);
          InsertOrdered(y, tail); } // ind. hyp.
        Ordered(Insert(y, tail));
    ==> { assert x <= y ==>
          Insert(y, Cons(x, tail)) == Cons(x, Insert(y, tail)); }
        Ordered(Insert(y, Cons(x, tail)));
    }
  }
}

```

Now we are in a position to prove the soundness of InsertSort using lemma InsertOrdered in a key step of the proof.

```

lemma {:induction false} InsertSortOrdered(xs: List<int>)
  ensures  Ordered(InsertSort(xs))
{
  match xs
  case Nil =>
  case Cons(x, xs) =>
    calc {
      true;
    }
    ==> { InsertSortOrdered(xs); } // ind. hyp.
        Ordered(InsertSort(xs));
    ==> { InsertOrdered(x, InsertSort(xs)); }
        Ordered(Insert(x, InsertSort(xs)));
    ==> Ordered(InsertSort(Cons(x, xs)));
  }
}

```

Proving that InsertSort always produces an ordered list does not prove it's a sorting

algorithm: in fact, the returned list could have missed an element from the original list, added a new one, or even be the empty list Nil.

What is missing is the requirement that the output list must contain the same elements as the original one, specifically, it should be a permutation of it. To avoid complications, we follow Leino's proof based on the concept of projecting a list onto an integer.

The function `Project(xs, p)` returns a list of all occurrences of `p` in `xs`, with each occurrence of `p` included the same number of times.

```
ghost function Project(xs: List<int>, p: int): List<int>
{
  match xs
  case Nil => Nil
  case Cons(x, tail) =>
    if x == p then Cons(x, Project(tail, p))
    else Project(tail, p)
}
```

Now, since the parameters in a lemma represent arbitrary values and are therefore implicitly universally quantified, to prove that `InsertSort(xs)` is a permutation of `xs`, it suffices to show that

```
Project(xs, p) == Project(InsertSort(xs), p)
```

with `p` as a parameter in the lemma. This can be done by using the lemma:

```
lemma InsertSameEl(y: int, xs: List<int>, p: int)
  ensures Project(Cons(y, xs), p) == Project(Insert(y, xs), p)
{}

lemma InsertSortSameEl(xs: List<int>, p: int)
  ensures Project(xs, p) == Project(InsertSort(xs), p)
{
  match xs
  case Nil =>
  case Cons(x, tail) =>
    InsertSameEl(x, InsertSort(tail), p);
}
```

References: Leino, chapters 4, 6, and 7, and chapter 8, paragraphs 8.0 - 8.1.